# The PARDISO sparse direct solver for *Úa*

Installation and user guide

October 2026

**Abstract**

*Úa* can solve the linear systems of its Newton–Raphson iterations with PARDISO, the parallel sparse direct solver of the Intel oneAPI Math Kernel Library (oneMKL), instead of MATLAB's built-in sparse solver. This guide describes how to install oneMKL, how to make PARDISO available in *Úa*, how to activate it in a model run, and how to check that everything works. The installation has been tested on Windows, which is the main focus of the guide. Instructions for Linux are given as well, but the Linux installation has not yet been fully tested.

## Contents

**1 Introduction** **2**

**2 Quick start (Windows)** **3**

**3 Requirements (Windows)** **3**

**4 Installing oneMKL on Windows** **4**

**5 Making PARDISO available in *Úa*** **5**

**6 Using PARDISO in *Úa*** **7**

6.1 Activating PARDISO in a run . . . . . 7

6.2 Checking a run . . . . . 7

6.3 Options . . . . . 8

6.4 Safeguards . . . . . 9

6.5 Combining with the parallel assembly . . . . . 9

6.6 Number of cores used by PARDISO . . . . . 9

6.7 Measuring the performance on your computer . . . . . 10

**7 Performance comparison** **10**

7.1 Set-up . . . . . 11

7.2 Times . . . . . 11

7.3 Findings . . . . . 12

**8 Troubleshooting** **13**

**9 Updating and removing oneMKL** **14**

**10 Linux (not yet fully tested)** **14**

**11 macOS** **16**

**12 Limitations** **16**

**A Files** **17**

# 1 Introduction

In each Newton–Raphson iteration of the uv (diagnostic) and uvh (transient) solvers of *Úa*, a large, sparse linear system has to be solved. By default, *Úa* uses MATLAB's built-in sparse direct solver for this. As an alternative, *Úa* can use PARDISO, the sparse direct solver of Intel's oneMKL library. PARDISO gives the same results (to round-off), but is considerably faster for the systems of *Úa*:

- A sparse direct solver works in three phases: an *analysis* (reordering and symbolic factorisation), a *numerical factorisation*, and a *forward and back substitution*. MATLAB's solver does all three in every call. PARDISO allows the analysis to be reused, and *Úa* does so: on a given mesh the sparsity pattern of the matrices does not change, so the analysis, which is the most expensive of the three phases, is done only once per mesh.
- The numerical factorisation and the substitution, which are done in every Newton–Raphson iteration, run in parallel on all cores of the computer.

As a result, a solution of the linear system in a Newton–Raphson iteration is typically 7 to 10 times faster with PARDISO than with MATLAB's solver, and long transient runs, which consist mostly of such solves, take about half as long. Short runs gain less, as they spend a larger part of their time on setting up the model and writing files.

**What is needed.** *Úa* itself contains the interface to PARDISO: the folder `UaSource\UaPardiso` with a compiled MEX file (`pardisoMex`) and the MATLAB function `UaPardisoSolve.m`. What has to be installed in addition is oneMKL, which is freely available from Intel. This has to be done only once per computer. In a model run, PARDISO is then selected with a single line in `DefineInitialInputs.m`.

**Status.**

- **Windows:** tested (64-bit Windows, MATLAB R2026b, oneMKL 2026.1 and 2026.2).
- **Linux:** the interface compiles and works on Linux, but this has so far only been tested outside MATLAB. The Linux instructions in Section 10 are therefore likely, but not yet confirmed.
- **macOS:** not supported, as Intel provides no oneMKL for current versions of macOS.

## 2 Quick start (Windows)

For users who want to get started straight away; each step is described in detail in Sections 4 and 5. Replace `<user>` by your Windows user name, and `C:\...\UaSource` by the location of your *Úa* source folder.

**1. Install oneMKL**, in a Windows command window:

```
py -3 -m venv C:\Users\<user>\mkl_venv
C:\Users\<user>\mkl_venv\Scripts\pip install mkl
```

**2. Tell *Úa* where oneMKL is**, eg in `startup.m`:

```
setenv("UA_MKL_RT","C:\Users\<user>\mkl_venv\Library\bin\mkl_rt.3.dll")
```

**3. Put the *Úa* source folder, with all subfolders, on the MATLAB path:**

```
addpath(genpath("C:\...\UaSource"))
```

**4. Select PARDISO** in `DefineInitialInputs.m`:

```
CtrlVar.LinearSolver.Type="PARDISO" ;
```

Before the first production run, check the installation as described in Step 7 (Section 5).

## 3 Requirements (Windows)

- 64-bit Windows.
- MATLAB. The compiled MEX file supplied with *Úa* (`pardisoMex.mexw64`) has been tested with R2026b. If it does not work with your version of MATLAB, it can be compiled with a C compiler for MEX files (Step 5).
- Python 3, but only for installing oneMKL with `pip` (Steps 1 and 2). Python is not used by *Úa*. If Python is not installed, install it from https://www.python.org; the installer also installs the Python launcher `py` used below. Check with `py -3 --version` in a command window.
- Internet access during the installation, and about 1 GB of disk space for oneMKL.
- A version of *Úa* that contains the folder `UaSource\UaPardiso`.
- A computer with several cores. PARDISO also works on a single core, but its advantage is largest on computers with 4 to 16 cores or more.

# 4 Installing oneMKL on Windows

The simplest way of installing oneMKL is to install the `mkl` package from the Python Package Index into a Python virtual environment of its own. The virtual environment is simply a folder that contains the oneMKL libraries; *Úa* only needs the location of one library file in it.

### Step 1: Create a virtual environment

Open a Windows command window (Start menu, type `cmd`), and create the virtual environment in a permanent location, eg in your user folder:

```
py -3 -m venv C:\Users\<user>\mkl_venv
```

Choose the location with some care:

- Do not use a temporary folder (eg `AppData\Local\Temp`): Windows may clean it up, and PARDISO would then stop working.
- Avoid folders that are synchronised with cloud storage (OneDrive, Google Drive, Dropbox): the installation consists of thousands of files, and files that are only kept online may not be available when MATLAB loads them.

### Step 2: Install the mkl package

```
C:\Users\<user>\mkl_venv\Scripts\pip install mkl
```

This downloads and installs oneMKL and the libraries it depends on; it takes about a minute. It should end with a line similar to

```
Successfully installed ... intel-openmp-2026.2.0 mkl-2026.2.0 ...
```

### Step 3: Locate the library

*Úa* needs the full path of the oneMKL runtime library `mkl_rt`. Check that it is there:

```
dir C:\Users\<user>\mkl_venv\Library\bin\mkl_rt*.dll
```

For oneMKL 2026 the file is called `mkl_rt.3.dll`, for oneMKL 2021 to 2025 `mkl_rt.2.dll`. The full path, here

```
C:\Users\<user>\mkl_venv\Library\bin\mkl_rt.3.dll
```

is what *Úa* needs (Step 6). The library loads the other oneMKL libraries from the same folder, so leave the folder as it is: do not copy `mkl_rt.3.dll` on its own to another place.

**Alternative: Intel's installer.** oneMKL can also be installed with Intel's own installer (as part of the oneAPI toolkits, or as the stand-alone oneMKL package). *Úa* should then work with the `mkl_rt` library in the `bin` folder of that installation, but this has not been tested.

# 5 Making PARDISO available in *Úa*

### Step 4: Put the *Úa* source folder on the MATLAB path

The interface to PARDISO is in the folder `UaSource\UaPardiso`. It is on the MATLAB path if the *Úa* source folder is added with all its subfolders, eg

```
addpath(genpath("C:\...\UaSource"))
```

Check this in MATLAB with

```
which pardisoMex         % should show ...\UaSource\UaPardiso\pardisoMex.mexw64
which UaPardisoSolve     % should show ...\UaSource\UaPardiso\UaPardisoSolve.m
```

If you add only some of the *Úa* folders to the path, make sure that `UaPardiso` is one of them.

### Step 5: The MEX file (compile only if needed)

The compiled MEX file `pardisoMex.mexw64` is supplied with *Úa*. Compile it yourself only if `which pardisoMex` does not find it, or if MATLAB reports that it is not a valid MEX file:

1. Install a C compiler for MEX files, eg the MinGW-w64 compiler, which is available as a MATLAB add-on (Home tab, Add-Ons, search for "MinGW").
2. Select it with

   ```
   mex -setup C
   ```

3. Compile the interface with

   ```
   BuildPardisoMex
   ```

   which creates `pardisoMex.mexw64` in the folder `UaPardiso`.

No oneMKL header files or libraries are needed for compiling, as `pardisoMex` loads the oneMKL library only when it is first used.

### Step 6: Tell *Úa* where oneMKL is

*Úa* finds the oneMKL library through one of the following, in this order of precedence:

1. the option `CtrlVar.LinearSolver.PARDISO.Library`, if it is not empty, or
2. the environment variable `UA_MKL_RT`.

The environment variable is the most convenient, as it has to be set only once. There are two ways of setting it:

- In MATLAB, at the start of each session, eg in your `startup.m`, or in a MATLAB Favorite that you use to set up your *Úa* paths:

```
setenv("UA_MKL_RT","C:\Users\<user>\mkl_venv\Library\bin\mkl_rt.3.dll")
```

- Permanently, as a Windows user environment variable, in a command window:

```
setx UA_MKL_RT "C:\Users\<user>\mkl_venv\Library\bin\mkl_rt.3.dll"
```

  This applies to MATLAB sessions started afterwards, not to sessions that are already running.

Alternatively, give the path in `DefineInitialInputs.m` of each run:

```
CtrlVar.LinearSolver.PARDISO.Library=...
    "C:\Users\<user>\mkl_venv\Library\bin\mkl_rt.3.dll" ;
```

Check the environment variable in MATLAB with

```
getenv("UA_MKL_RT")              % should show the full path of mkl_rt.3.dll
isfile(getenv("UA_MKL_RT"))      % should be 1
```

### Step 7: Check the installation

In a new MATLAB session, with the *Úa* paths and `UA_MKL_RT` set, run:

```
pardisoMex('load',getenv("UA_MKL_RT"),'TBB') ;
disp(pardisoMex('version'))
n=300 ; A=gallery('poisson',n)+speye(n^2) ; A=A+0.1*triu(A,1) ; b=A*ones(n^2,1) ;
h=pardisoMex('init',11) ;
r=pardisoMex('factorp',h,A,A) ;
x=pardisoMex('solvep',h,b) ;
pardisoMex('free',h) ;
fprintf('max error %.1e\n',norm(x-1,inf)) ;
```

This loads oneMKL, and solves a test system with 90,000 unknowns whose solution is a vector of ones. It should print the oneMKL version and a very small error, eg

```
Intel(R) oneAPI Math Kernel Library Version 2026.2-Product Build 20260915
for Intel(R) 64 architecture applications
max error 3.3e-16
```

Then check the solver as *Úa* calls it:

```
CtrlVar=Ua2D_DefaultParameters() ; CtrlVar.LinearSolver.Type="PARDISO" ;
x=UaPardisoSolve(CtrlVar,A,b) ;
fprintf('max error %.1e\n',norm(x-1,inf)) ;
Stats=UaPardisoSolve("stats")
```

This should again print an error of the order of $10^{-15}$ or smaller, and statistics with one analysis, one factorisation, and no fallbacks.

Two notes:

- In calls of `pardisoMex`, the commands (`'load'`, `'init'`, ...) must be given in single quotes (character arrays), not double quotes (strings).
- The threading layer of oneMKL (`'TBB'` above, which is also the default in *Úa*) is fixed when oneMKL is first loaded in a MATLAB session. To use another layer, MATLAB has to be restarted (Section 6.3).

If any of the checks fails, see Section 8.

# 6 Using PARDISO in *Úa*

## 6.1 Activating PARDISO in a run

In `DefineInitialInputs.m`:

```
CtrlVar.LinearSolver.Type="PARDISO" ;
```

and, only if the environment variable `UA_MKL_RT` is not used (Step 6),

```
CtrlVar.LinearSolver.PARDISO.Library=...
    "C:\Users\<user>\mkl_venv\Library\bin\mkl_rt.3.dll" ;
```

No other changes are needed; the other options (Section 6.3) have suitable defaults. MATLAB's solver remains the default, `CtrlVar.LinearSolver.Type="MATLAB"`, so runs that do not set this option are not affected.

PARDISO is then used for the linear systems of the Newton–Raphson iterations of the uv and uvh solvers, and for other systems solved through the same functions (`solveKApe.m` and `solveKApeSymmetric.m`). Small systems, with fewer than 20,000 unknowns, are still solved with MATLAB's solver, for which they are fast enough.

## 6.2 Checking a run

Before using PARDISO for production runs of a new model, do a short run (eg a few time steps from a restart file) once with `CtrlVar.LinearSolver.Type="MATLAB"` and once with `"PARDISO"`, and compare:

- the number of Newton–Raphson iterations should be the same,
- the results should agree to round-off, and
- the run with PARDISO should be faster.

At the end of a run with PARDISO, in the same MATLAB session,

```
Stats=UaPardisoSolve("stats")
```

gives the following counts since the start of the session (or since the last `UaPardisoSolve ("reset")`):

| field | meaning |
|---|---|
| `Analyses` | number of analyses |
| `Factorisations`, `Solves` | numbers of numerical factorisations and of solves |
| `StructuralPatterns` | number of times the sparsity pattern of a mesh was given to PARDISO (normally once per mesh and type of system) |
| `PatternExtensions` | number of extensions of a sparsity pattern (normally none) |
| `Resolves` | solves repeated after a new analysis, because of a too large residual |
| `Fallbacks` | systems solved with MATLAB's solver after the residual check failed |
| `SPDFallbacks` | Cholesky factorisations that failed, solved as symmetric indefinite instead |
| `SmallSystems`, `DiagonalSystems` | systems solved with MATLAB's solver because they were small or diagonal |
| `States`, `StatesReleased` | current number of PARDISO states, and number released |
| `MaxResidual` | largest relative residual of all solutions |

In a normal run, there is about one analysis per mesh and type of system (eg one for the uvh system of a transient run, plus one for the uv system of an initial diagnostic solve), many factorisations, and no fallbacks. After remeshing, one new analysis per new mesh is expected.

`UaPardisoSolve("reset")` releases all memory held by PARDISO and resets the counts. The factors of large systems can require considerable memory; at most six PARDISO states are kept at any time.

### 6.3 Options

The options are fields of `CtrlVar.LinearSolver`. Like all other options, they can be changed in `DefineInitialInputs.m`; their defaults are set in `Ua2D_DefaultParameters.m`.

| field | default | meaning |
|---|---|---|
| `Type` | `"MATLAB"` | `"MATLAB"` or `"PARDISO"` |
| `PARDISO.Library` | `""` | full path of `mkl_rt`; if empty, the environment variable `UA_MKL_RT` is used |
| `PARDISO.ThreadingLayer` | `"TBB"` | `"TBB"` or `"SEQUENTIAL"` |
| `PARDISO.ResidualTolerance` | $10^{-10}$ | tolerance of the residual check |
| `PARDISO.MinimumSize` | 20000 | systems with fewer unknowns are solved with MATLAB's solver |
| `PARDISO.StructuralPattern` | `true` | use the fixed sparsity pattern of the mesh |

**Threading layer.** With `"TBB"` (the default), oneMKL uses all cores available to MATLAB, through the TBB library that MATLAB itself uses. `"SEQUENTIAL"` uses a single core; it is mainly useful as a fall-back if the TBB layer causes problems, or if several MATLAB processes share a computer and each should use only one core. The OpenMP layer of oneMKL can not be used within MATLAB and is rejected. The threading layer is fixed when oneMKL is first loaded in a MATLAB session; to change it, restart MATLAB.

**Minimum size.** For small systems, the overheads of PARDISO cost more than they save. With the default, PARDISO is used for the uv and uvh systems of all but small meshes.

**Structural pattern.** With the default `true`, PARDISO is given the sparsity pattern of all matrices that can occur on the mesh, so that the analysis is done only once per mesh. There is normally no reason to change this.

**Residual tolerance.** Each solution is checked (Section 6.4); there is normally no reason to change the tolerance.

### 6.4 Safeguards

`UaPardisoSolve` checks every solution, so that a problem with PARDISO can not go unnoticed:

- The relative residual of each solution is computed. If it is above `ResidualTolerance`, the analysis is repeated and the system solved again; if the residual is still too large, the system is solved with MATLAB's solver, and the warning `UaPardisoSolve:Residual` is given. The run continues correctly.
- If a Cholesky factorisation of a matrix expected to be symmetric positive definite fails (eg a Hessian in an inversion), the system is solved as symmetric indefinite instead, with the warning `UaPardisoSolve:NotPositiveDefinite`. The solution is correct.
- Small and diagonal systems are solved with MATLAB's solver.

### 6.5 Combining with the parallel assembly

PARDISO speeds up the solution of the linear systems; the parallel assembly of *Úa* speeds up the assembly of the residuals and the Jacobians. The two gains add up, and the two do not compete for the cores: PARDISO runs in the MATLAB client and uses all cores while the workers of the parallel pool are idle. A thread-based pool with 4 workers has worked well together with PARDISO on computers with 16 and 32 cores:

```
parpool("Threads",4) ;                                  % before the run
CtrlVar.Parallel.uvhAssembly.spmd.isOn=true ;           % in DefineInitialInputs.m
CtrlVar.Parallel.uvAssembly.spmd.isOn=true ;
CtrlVar.LinearSolver.Type="PARDISO" ;
```

The functions `uvhAssemblyScalingTest.m` and `uvAssemblyScalingTest.m` help to choose the number of workers for a given model and computer.

### 6.6 Number of cores used by PARDISO

With the default threading layer, PARDISO uses all cores available to the MATLAB process. The number of cores can only partly be controlled:

- There is no option in *Úa* for the number of cores, and neither the oneMKL function `MKL_Set_Num_Threads` nor the environment variable `MKL_NUM_THREADS` has any effect with the TBB layer.
- A single core: `CtrlVar.LinearSolver.PARDISO.ThreadingLayer="SEQUENTIAL"`.
- A chosen number of cores: start MATLAB itself restricted to these cores (processor affinity, see below).

**Processor affinity.** Every process has a processor affinity: the set of logical processors on which the operating system may run its threads. A process can be started with a smaller set; all its threads then only ever run on these processors. MATLAB and the TBB library determine the number of cores when they start, so a MATLAB session started with an affinity of $p$ cores behaves much like MATLAB on a computer with $p$ cores: `feature("numcores")` returns $p$, and MATLAB's own threads, PARDISO, and the workers of a thread-based pool all use only these cores. On Windows, start MATLAB from a command window with, eg

```
start "" /affinity 5555 "C:\Program Files\MATLAB\R2026b\bin\matlab.exe"
```

The hexadecimal mask selects the logical processors (bit $i$ for processor $i$). With hyperthreading, Windows usually numbers the two logical processors of a core consecutively, so that every other bit selects one logical processor per core: `5555` selects 8 cores, `55555555` 16. Without hyperthreading, `FF` selects 8 cores.

Two points should be noted. The affinity restricts the process to these processors, but does not reserve them for it: other programs can still run on them. And the restriction has to be in place when MATLAB starts: do not change the affinity of a running MATLAB session (eg with "Set affinity" in the Task Manager), as MATLAB and TBB do not adapt to it, and their threads then compete for the remaining cores, which makes PARDISO considerably slower.

Limiting the number of cores is mainly useful when several runs are done at the same time on one computer, eg four runs with 8 cores each on a computer with 32 cores. The speed-up of PARDISO levels off beyond about 8 to 16 cores, so this gives more throughput in total.

### 6.7 Measuring the performance on your computer

`LinesolveScalingTest.m` measures the time of the linear solves with MATLAB's solver and with PARDISO, as a function of the number of cores, for the uvh (or uv) system of a given restart file:

```
Results=LinesolveScalingTest("MyRun-RestartFile.mat") ;
```

It times the three phases of PARDISO separately, both with a new analysis for each solve and with the analysis reused (as in *Úa*), and also the solves as called in the Newton–Raphson iterations of *Úa*. Each measurement is a separate MATLAB process restricted to a given number of cores, so the test takes a few minutes; the computer should be otherwise idle. See `help LinesolveScalingTest` for the options.

`LinesolveSizeScalingTest.m` does the same for a set of restart files of the same model with different mesh resolutions, and gives the times of the phases as functions of the number of unknowns, eg

```
Results=LinesolveSizeScalingTest("C:\...\MyRun-nx*-RestartFile.mat") ;
```

Section 7 summarises the results of these tests for a Greenland model.

## 7 Performance comparison

This section summarises measurements of the time needed to solve the linear uvh system of a Newton–Raphson iteration, with MATLAB's solver and with PARDISO, for meshes of different

resolution. It shows where PARDISO's advantage comes from, and how it depends on the size of the problem and on the number of cores.

### 7.1 Set-up

The systems are those of a transient model of the Greenland ice sheet on seven meshes (Table 1), from 870 to 465,882 unknowns, taken from restart files and built as in *Úa* (the uvh system, after the elimination of the active constraints). MATLAB solves such a system with its LU factorisation (UMFPACK). For PARDISO, the three phases were timed separately (analysis, numerical factorisation, forward and back substitution), once with a new analysis for each solve, as MATLAB's solver always does, and once with the analysis reused, as in *Úa*. All measurements were done with `LinesolveSizeScalingTest.m` (Section 6.7) on a Windows computer with two Intel Xeon Silver 4208 processors with 8 cores each (16 cores), MATLAB R2026b and oneMKL 2026.2, once restricted to a single core and once with all 16 cores (Section 6.6). The results apply to this computer; the test can be repeated on others.

### 7.2 Times

Table 1: Times of the solution of the uvh system on 16 cores, and the ratio of the time of MATLAB's solver to that of PARDISO (above 1: PARDISO is faster). "Reused": numerical factorisation and substitution, with the analysis of an earlier solve reused, as in *Úa*.

| | | | PARDISO | | | MATLAB | MATLAB / PARDISO | |
|---|---|---|---|---|---|---|---|---|
| nx | nodes | unknowns | analysis | reused | all phases | | all phases | reused |
| 10 | 290 | 870 | 0.003 s | 0.002 s | 0.005 s | 0.006 s | 1.1 | 3.2 |
| 30 | 2,314 | 6,942 | 0.037 s | 0.010 s | 0.049 s | 0.062 s | 1.3 | 6.5 |
| 50 | 6,324 | 18,972 | 0.126 s | 0.026 s | 0.158 s | 0.187 s | 1.2 | 7.1 |
| 100 | 25,095 | 75,285 | 0.563 s | 0.101 s | 0.680 s | 0.930 s | 1.4 | 9.2 |
| 150 | 56,070 | 168,210 | 1.364 s | 0.231 s | 1.636 s | 2.464 s | 1.5 | 10.7 |
| 200 | 99,370 | 298,110 | 2.614 s | 0.412 s | 3.104 s | 4.709 s | 1.5 | 11.4 |
| 250 | 155,294 | 465,882 | 3.897 s | 0.702 s | 4.703 s | 7.186 s | 1.5 | 10.2 |

Table 1 gives the times on 16 cores, and Figure 1 the times of all phases on one and on 16 cores. Table 2 gives the exponents $\alpha$ of power laws $T = c\,n^{\alpha}$ fitted to the five systems with 18,972 to 465,882 unknowns; all fits are very close ($R^2 \geq 0.998$).

Table 2: Exponents $\alpha$ of power laws $T = c\,n^{\alpha}$ fitted to the systems with 18,972 to 465,882 unknowns. For comparison, over this range $n \log n$ corresponds to an exponent of 1.09, and $n^{1.5}$ to 1.50.

| | 1 core | 16 cores |
|---|---|---|
| PARDISO: analysis | 1.11 | 1.08 |
| PARDISO: numerical factorisation | 1.20 | 1.02 |
| PARDISO: forward and back substitution | 1.03 | 0.97 |
| PARDISO: solve with the analysis reused | 1.19 | 1.02 |
| PARDISO: all three phases | 1.14 | 1.07 |
| MATLAB's solver | 1.17 | 1.15 |
| nonzeros in PARDISO's factors (fill-in) | 1.15 | |

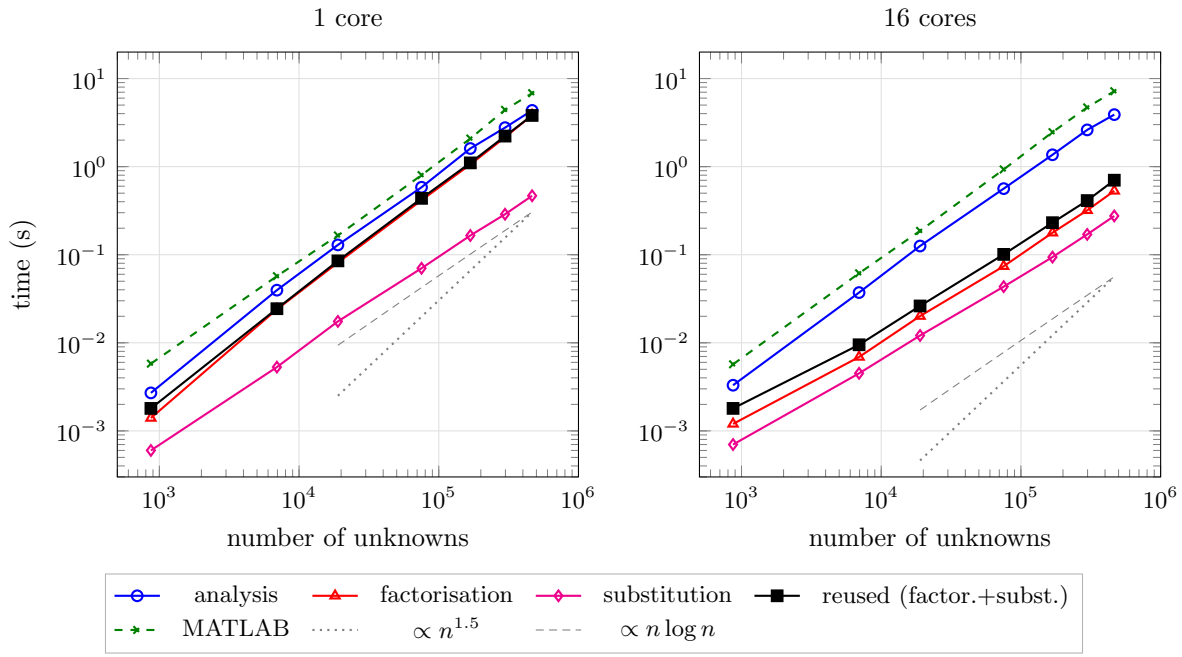



Figure 1: Times of the phases of PARDISO and of MATLAB's solver as functions of the number of unknowns, on one core (left) and on 16 cores (right). "Factorisation" and "substitution" are the phases after a new analysis; "reused" is a solve (numerical factorisation and substitution) with the analysis reused, as in *Úa*. The grey lines show the growth $\propto n^{1.5}$ and $\propto n \log n$ for comparison (their vertical position is arbitrary).

### 7.3 Findings

1. **With all three phases, PARDISO is not much faster than MATLAB's solver.** On 16 cores it is 1.2 to 1.5 times faster; on a single core it is even slower (MATLAB / PARDISO between 0.73 and 0.85 for 18,972 unknowns and more).

2. **The large gain comes from reusing the analysis.** The analysis is the most expensive phase: for the largest system, half of a solve with all three phases on one core, and 83% on 16 cores. With the analysis reused, PARDISO is about twice as fast as MATLAB's solver on one core, and 7 to 11 times as fast on 16 cores. In *Úa* the analysis is done only once per mesh, so this is the speed-up that matters in practice.

3. **The phases differ in how they use the cores.** The numerical factorisation makes good use of the cores, and the better the larger the system: it is 4 times faster on 16 cores than on one for 18,972 unknowns, and 7 times faster for 465,882 unknowns. The analysis hardly benefits from more cores, and the substitution, which is limited by memory bandwidth, only to a small extent. MATLAB's solver does not get faster with more cores at all.

4. **How the time grows with the size of the problem.** On a single core, the phases grow as expected for two-dimensional meshes: the analysis and the fill-in about as $n \log n$, the factorisation somewhat faster (exponent 1.20, still well below the asymptotic $n^{1.5}$, which does not dominate at these sizes), and the substitution about linearly. On 16 cores, the increasing benefit of the cores for larger systems almost exactly offsets the growth of the factorisation, so that a solve with the analysis reused grows about linearly with $n$ in this range: doubling the number of unknowns roughly doubles the time of a solve. The time of MATLAB's solver grows somewhat faster (exponent 1.15), so that PARDISO's advantage grows with the size of the problem, from about 7 times for 19,000 unknowns to 10 to 11 times for 170,000 unknowns and more.

5. **Small systems.** PARDISO is also faster than MATLAB's solver for small systems when the analysis is reused (eg 6.5 times for 6,942 unknowns on 16 cores). By default, *Úa* nevertheless uses MATLAB's solver for systems with fewer than 20,000 unknowns (`CtrlVar.LinearSolver.PARDISO.MinimumSize`), as the overheads of `UaPardisoSolve` (pattern handling, residual check), which are not included in these measurements, are relatively larger for small systems, and the times are short anyway.

**What this means in practice.** PARDISO pays off most in runs with many Newton–Raphson iterations on the same mesh, eg transient runs over many time steps, on computers with several cores, and its advantage grows with the size of the mesh. For a ten-year transient run of the Greenland model with 99,370 nodes, the run time was about halved with PARDISO (44 instead of 87 minutes, on a computer with 32 cores), as the linear solves were about seven times faster and made up most of the run time with MATLAB's solver. Runs with few Newton–Raphson iterations, or on a single core, gain less.

# 8 Troubleshooting

For each message or symptom, the cause and the remedy:

- `The MKL runtime library (mkl_rt) was not found. Set CtrlVar.LinearSolver.PARDISO.Library, or ...`
  Neither `CtrlVar.LinearSolver.PARDISO.Library` nor `UA_MKL_RT` gives the path of an existing file. Check the path (Steps 3 and 6). Note that `setx` only applies to MATLAB sessions started afterwards.

- `Undefined function 'UaPardisoSolve'` or `'pardisoMex'`
  `UaSource\UaPardiso` is not on the MATLAB path (Step 4).

- `Invalid MEX-file` or a similar message for `pardisoMex`
  The supplied MEX file does not work with your version of MATLAB. Compile it (Step 5).

- `Could not load ... (Windows error 126)`
  A library that `mkl_rt` needs was not found, usually because `mkl_rt.3.dll` was copied out of its folder, or because the folder is in cloud storage and not all files are available locally. Use the path in the original oneMKL folder (Steps 1 to 3).

- `MKL is already loaded with another threading layer; restart MATLAB to change it.`
  The threading layer was changed within a MATLAB session. Restart MATLAB.

- `layer must be "TBB" or "SEQUENTIAL" (the OpenMP layer can not be used within MATLAB).`
  Use one of these two layers.

- `First argument must be a command string.`
  A command of `pardisoMex` was given in double quotes; use single quotes, eg `pardisoMex('version')`.

- Warning `UaPardisoSolve:Residual`
  A solution did not pass the residual check even after a new analysis, and was computed with MATLAB's solver. The run continues correctly. If this happens often, please report it, and use `CtrlVar.LinearSolver.Type="MATLAB"` in the meantime.

- Warning `UaPardisoSolve:NotPositiveDefinite`
  A matrix expected to be positive definite was not; it was solved as symmetric indefinite instead. The solution is correct.

- `Reusing a decomposition (modified Newton-Raphson) is not supported`
  Reusing factorisations over several Newton–Raphson iterations is not supported with PARDISO; switch it off (it is off by default).

- No or little speed-up
  The run does few Newton–Raphson iterations, so that other parts of the run (setting up the model, writing files) dominate; the systems are smaller than `MinimumSize` and solved with MATLAB's solver; the `"SEQUENTIAL"` layer is used; MATLAB was started with a restricted processor affinity; or the computer has few cores. `UaPardisoSolve("stats")` shows how many systems were solved with PARDISO.

- PARDISO stops working after some time
  The oneMKL folder was removed, eg because it was in a temporary folder. Reinstall oneMKL in a permanent location (Step 1).

# 9 Updating and removing oneMKL

**Updating.** To update oneMKL to the latest version, in a command window:

```
C:\Users\<user>\mkl_venv\Scripts\pip install --upgrade mkl
```

Then check the name of the library (Step 3): with a new major version of oneMKL, the number in `mkl_rt.3.dll` may change, and the path in `UA_MKL_RT` has to be changed accordingly. Restart MATLAB afterwards.

**Removing.**

1. Remove `CtrlVar.LinearSolver.Type="PARDISO"` from your runs, or set it to `"MATLAB"`.
2. Remove the environment variable: delete the `setenv` line from `startup.m`, or, if it was set with `setx`, delete it under "Edit environment variables for your account" (Start menu), or with

   ```
   reg delete HKCU\Environment /v UA_MKL_RT /f
   ```

3. Delete the folder `C:\Users\<user>\mkl_venv`.

# 10 Linux (not yet fully tested)

**Status:** the interface `pardisoMex.c` compiles and works on Linux, but this has so far only been tested outside MATLAB (Ubuntu 24.04, gcc 13, oneMKL 2026.2), not yet within MATLAB. The steps below are therefore likely to work, but have not yet been confirmed. Please report your experience.

### L1: Install oneMKL

```
python3 -m venv ~/mkl_venv
~/mkl_venv/bin/pip install mkl tbb
```

The library is then `~/mkl_venv/lib/libmkl_rt.so.3` (`libmkl_rt.so.2` for oneMKL 2021 to 2025). Check with

```
ls ~/mkl_venv/lib/libmkl_rt.so*
```

The package `tbb` is only needed if MATLAB's own TBB library is not found. As on Windows, leave the folder as it is, and do not copy the library on its own to another place.

### L2: Compile the interface

No compiled MEX file for Linux is supplied. With a C compiler supported by your version of MATLAB (gcc), in MATLAB:

```
mex -setup C
BuildPardisoMex
```

This creates `pardisoMex.mexa64` in the folder `UaPardiso`. As on Windows, no oneMKL header files or libraries are needed.

### L3: Tell *Úa* where oneMKL is

Set the environment variable before MATLAB is started, eg in `~/.bashrc` or in a job script:

```
export UA_MKL_RT=$HOME/mkl_venv/lib/libmkl_rt.so.3
```

or in MATLAB, eg in `startup.m`, with `setenv("UA_MKL_RT",...)`, or give the path in `CtrlVar.LinearSolver.PARDISO.Library`.

### L4: Check the installation

Do the check of Step 7, but first with the `'SEQUENTIAL'` layer, in a MATLAB session that is allowed to fail (eg a session started only for this check):

```
pardisoMex('load',getenv("UA_MKL_RT"),'SEQUENTIAL') ;
```

followed by the remaining commands of Step 7. If this works, repeat the check in a new MATLAB session with `'TBB'`. Then compare a short run with MATLAB's solver and with PARDISO (Section 6.2). If the TBB layer causes problems, use `CtrlVar.LinearSolver.PARDISO.ThreadingLayer="SEQUENTIAL"`.

### Notes for Linux

- On Linux, all loaded libraries share a common symbol namespace, so that oneMKL could inadvertently call functions of the (reduced) MKL that MATLAB itself has loaded. `pardisoMex` therefore loads oneMKL in a way that makes it use its own functions, and loads the other oneMKL libraries by their full path from the folder of `libmkl_rt`. If MATLAB crashes when oneMKL is loaded or at the first factorisation, the most likely cause is a conflict with MATLAB's own MKL or TBB libraries; try the `'SEQUENTIAL'` layer.
- Do not add the oneMKL folder to `LD_LIBRARY_PATH`, and do not use `LD_PRELOAD`, to get around such problems: that would affect MATLAB's own libraries as well.
- On clusters, a job is usually restricted to the cores allocated to it by the scheduler; PARDISO then uses these cores. On a workstation, MATLAB can be restricted to some of the cores with `taskset`, eg `taskset -c 0-7 matlab`. Linux usually numbers the physical cores first and the hyperthreads after them; `lscpu -p` shows which logical processors belong to which core.
- A job script for a batch run might look like

```
export UA_MKL_RT=$HOME/mkl_venv/lib/libmkl_rt.so.3
matlab -batch "MyUaRunScript"
```

## 11 macOS

PARDISO is not available on macOS, as Intel provides no oneMKL for current versions of macOS. On macOS, *Úa* uses MATLAB's solver.

## 12 Limitations

- PARDISO is used for the systems solved through `solveKApe.m` and `solveKApeSymmetric.m`. Some of their solution methods (eg for constraints that are not nodal, and the iterative solvers) and factorisations done elsewhere (eg of the mass matrix) use MATLAB's solver.
- The fixed sparsity pattern of the mesh is used by the uv and uvh solvers. Other callers, eg in the inversion, use a pattern that PARDISO learns from the matrices, which can require a few more analyses.
- Reusing a factorisation over several Newton–Raphson iterations (modified Newton–Raphson) is not supported with PARDISO.
- The number of cores used by PARDISO can not be set within MATLAB (Section 6.6).
- Windows is tested; Linux is not yet fully tested; macOS is not supported.

# A Files

| file | content |
|---|---|
| `UaPardiso/pardisoMex.c` | MEX interface to PARDISO; loads `mkl_rt` at run time |
| `UaPardiso/pardisoMex.mexw64` | compiled interface (Windows) |
| `UaPardiso/BuildPardisoMex.m` | compiles `pardisoMex.c` (Windows and Linux) |
| `UaPardiso/UaPardisoSolve.m` | solves a system with PARDISO, with the checks, fallbacks and statistics |
| `UaMain/solveKApe.m`, `UaMain/solveKApeSymmetric.m` | solution of the linear systems with constraints; use PARDISO if selected |
| `UaMain/Ua2D_DefaultParameters.m` | defaults of the options `CtrlVar.LinearSolver.*` |
| `UaMain/LinesolveScalingTest.m` | performance of the linear solvers as a function of the number of cores |
| `UaMain/LinesolveSizeScalingTest.m` | performance of the phases of PARDISO and of MATLAB's solver as a function of the number of unknowns |